

上机辅导

程序调试初步(VS2010)

程序调试要点

- breakpoint(断点)
- execution(调试运行)
- watch data(查看数据)

程序调试示例

输入一个整数 n (假定 $n < 10$)，如果 n 大于0，则调用一个函数prt。
在prt中，输入 n 个整数，并存入数组 a ，然后按照反向顺序输出数组中的非负整数。

要求每行输出三个数，并且排列整齐。

例如， n 为9，且各个数据为：

1 0 3 5 6 0 6 3 2

则数组元素 $a[0]$ 到 $a[8]$ 的值为：

$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	$a[6]$	$a[7]$	$a[8]$	$a[9]$
1	0	3	5	6	0	6	3	2	

又例如， n 为8，且各个数据为：

0 2 3 5 2 -14 4 9

则数组元素 $a[0]$ 到 $a[7]$ 的值为：

$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	$a[6]$	$a[7]$	$a[8]$	$a[9]$
0	2	3	5	2	-14	4	9		

➤预期的程序运行结果

Input n: 9

1 0 3 5 6 0 6 3 2

2 3 6

0 6 5

3 0 1

Input n: 8

0 2 3 5 2 -14 4 9

9 4 2

5 3 2

0

Input n: 0

以上的斜体数字表示键入的数据。

➤程序

```
#include <stdio.h>
#include <stdlib.h> /* exit需要 */
void prt(short n)
{
    short a[10], i, j;
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    for(i=n-1, j=0; i>=0; i--)
        if(a[i] > 0)
        {
            j++;
            printf("%d", a[i]);
            printf("%c", j%3 ? '\n' : '\t');
        }
    if((j % 3) == 0)
        printf("\n");
}
```

```
main()
{
    short n;
    while(1)
    {
        printf("Input n[0-10]: ");
        scanf("%d", &n);
        if(n <= 0)
            exit(0);
        else if(n < 10)
            prt(n);
    }
}
/*      准备好的输入数据
9
1 0 3 5 6 0 6 3 2
8
0 2 3 5 2 -14 4 9
0
*/
```

➤编译和调试过程示例

1) 新建项目

启动VS，新建项目，添加源文件“cas4-7程序调试.c”。

2) 编译和生成解决方案(链接)

先单击编译按钮“”，或快捷键**Ctrl+F7**。

再单击链接按钮“”，或快捷键**F7**。

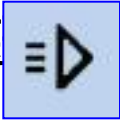
或者

一次单击链接按钮“”，或快捷键**F7**。

如果没有产生错误，表示完成编译和链接。

```
1>----- 已启动生成: 项目: cas4-7调试程序, 配置: Debug Win32 -----
1>生成启动时间为 2014/10/13 18:34:17。
1>ClCompile:
1> cas4-7调试程序.c
1>
1>生成成功。
1>
1>已用时间 00:00:00.29
===== 生成: 成功 1 个, 失败 0 个, 最新 0 个, 跳过 0 个 =====
```

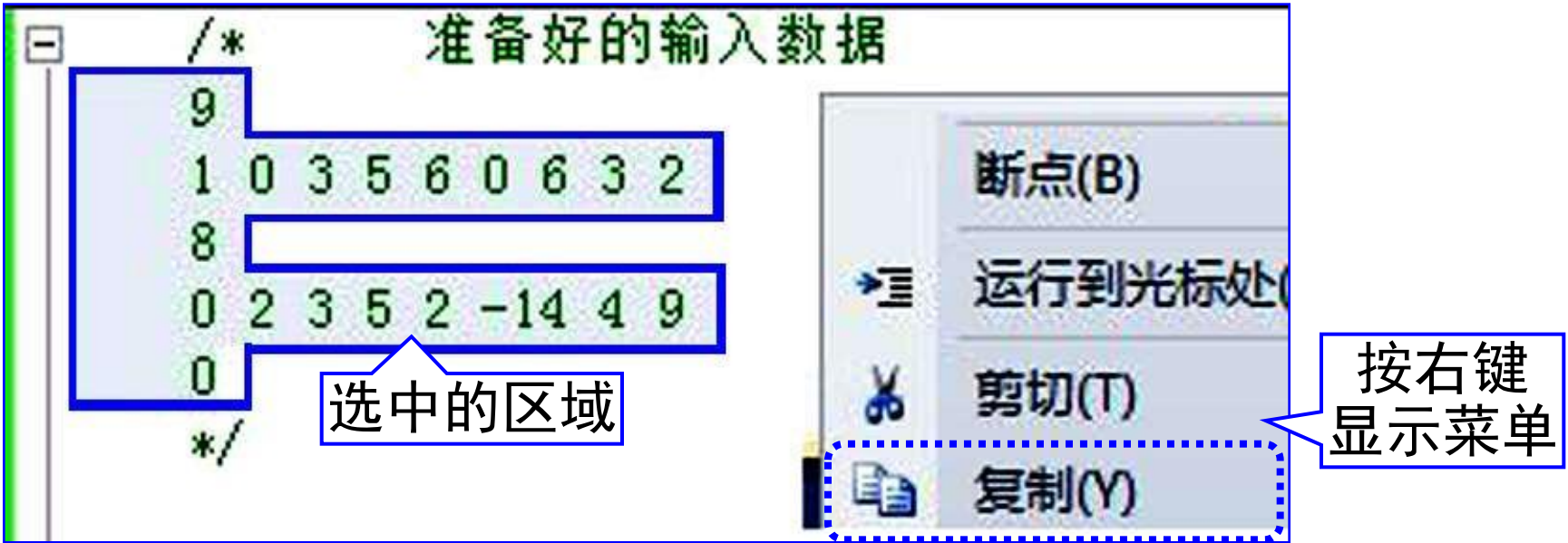
3) 执行程序

用菜单命令选择“调试(D)”->“开始执行(不调试)(H)”，或者左键单击程序运行按钮”，或使用快捷键**Ctrl+F5**。

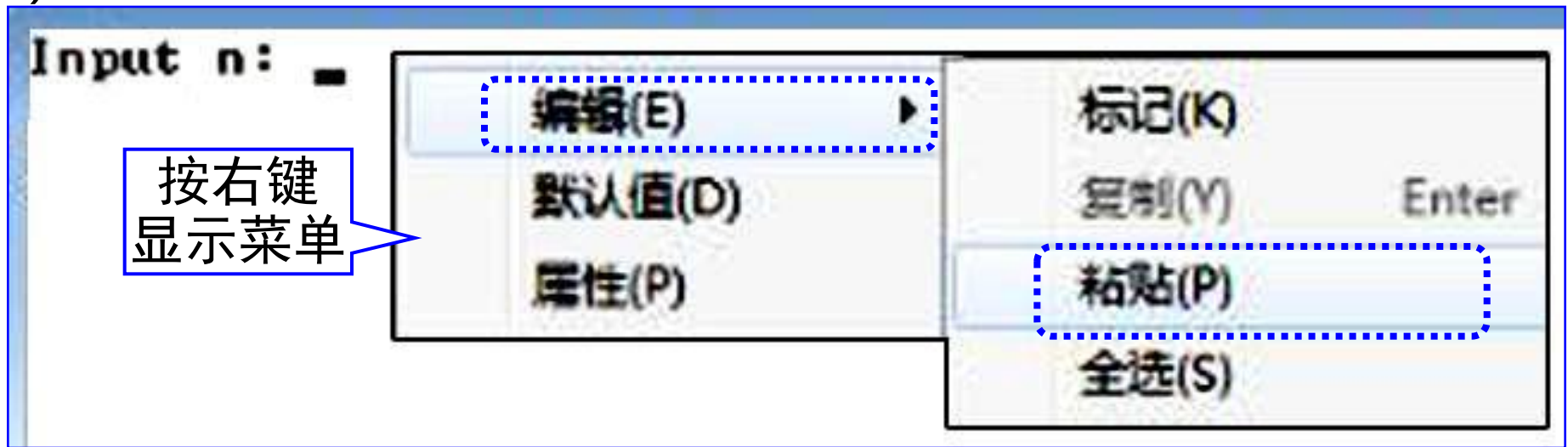


4) 复制准备好的输入数据

选中程序后面准备好的输入数据(用鼠标刷一下)，复制这些数据。



5) 将复制的输入数据粘贴到运行程序的窗口



6) 运行程序

观看程序运行的结果：

```
Input n: 9
1 0 3 5 6 0 6 3 2
2
3
6      6
5
3      1
Input n: 8
0 2 3 5 2 -14 4 9
9
4
2      5
3
2
Input n: 0
Press any key to continue
```

输出7个数据
没有输出0
换行不对

输出6个数据
没有输出0和-14
换行不对

预期的程序运行结果为：

```
Input n: 9
1 0 3 5 6 0 6 3 2
2      3      6
0      6      5
3      0      1
Input n: 8
0 2 3 5 2 -14 4 9
9      4      2
5      3      2
0
Input n: 0
Press any key to continue
```

输出3行
9个数据

输出3行
8个数据

与预期结果比较，发现错误：
输出数据的个数和换行的位置
都有错。

7) 分析程序出错的原因

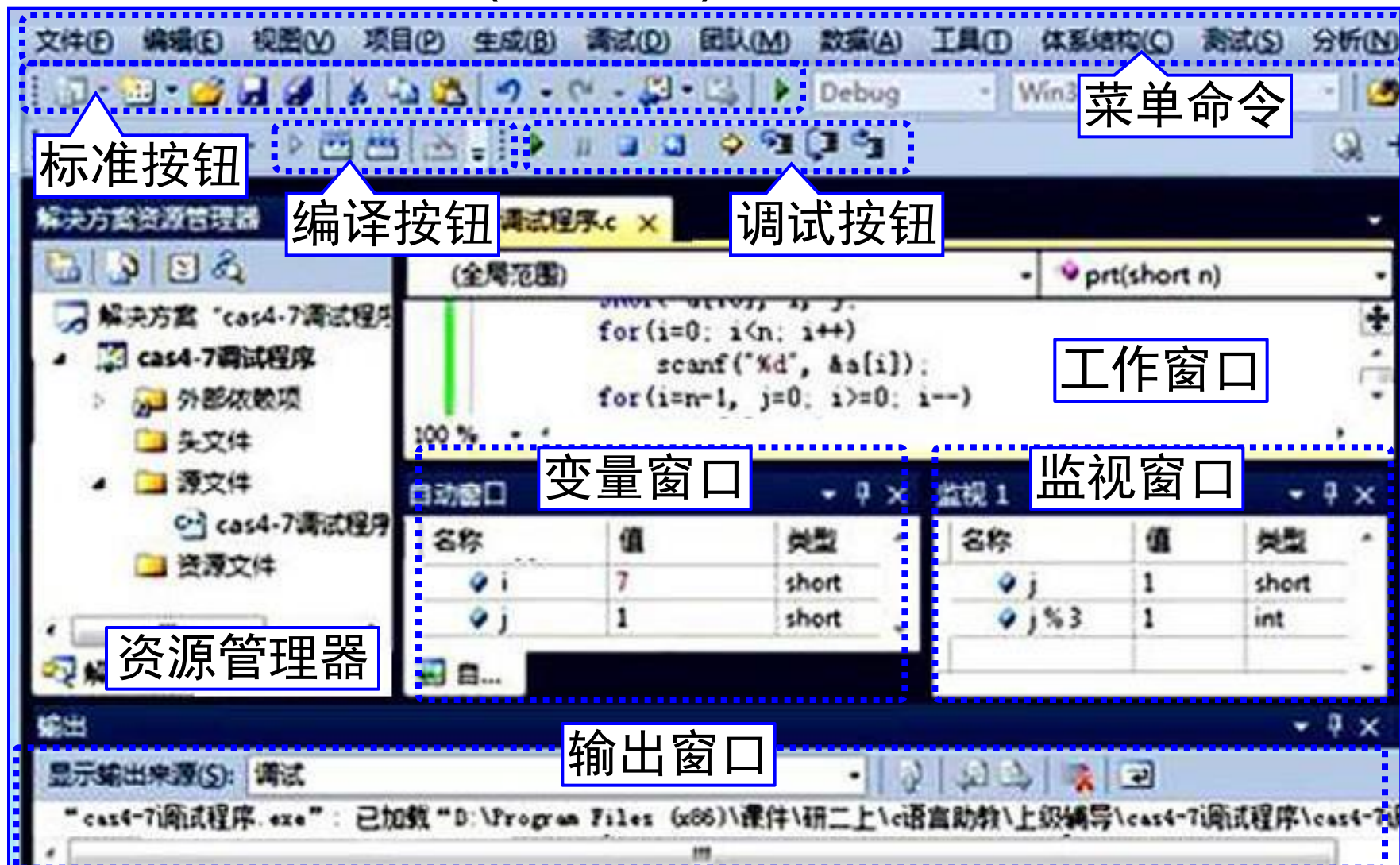
先分析输出格式问题。确定观察数据的位置在printf语句，准备在printf语句处设置断点。

```
#include <stdio.h>
#include <stdlib.h> /* exit需要的头文件 */
void prt(short n)
{
    short a[10], i, j;
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    for(i=n-1, j=0; i>=0; i--)
        if(a[i] > 0)
        {
            j++;
            printf("%d", a[i]);
            printf("%c", j%3 ? '\n' : '\t');
        }
    if((j % 3) == 0)
        printf("\n");
}
```

准备设置
断点的行

➤ 调试运行

VS在调试时将屏幕分成各种区域和窗口。常用的包括：菜单命令(固定)、标准按钮、编译按钮、调试按钮，工作窗口、变量窗口、监视窗口(观察窗口)、资源管理器、输出窗口等。

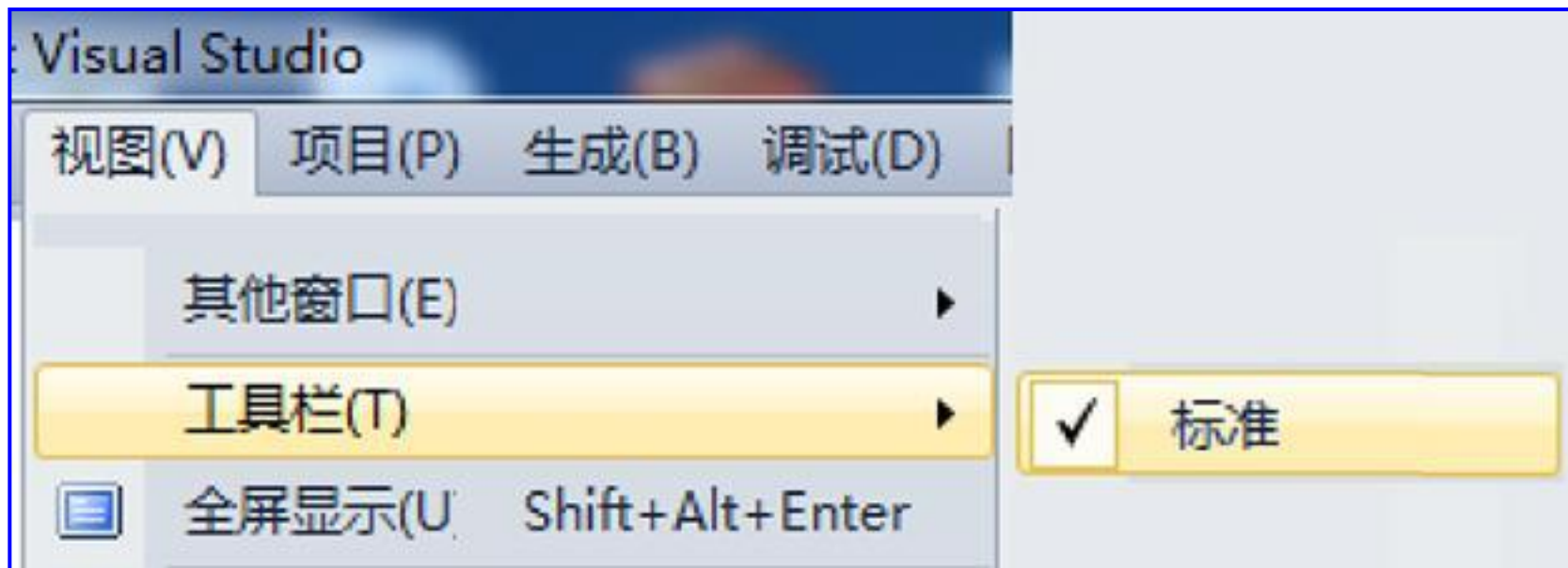


➤ 激活命令按钮的视图

菜单命令在VS中是固定的，标准按钮、编译按钮和调试按钮的视图可能在启动VS时没有出现，需要激活。

● 激活标准命令按钮的视图

用菜单命令选择“视图(V)”->“工具栏(T)”->“标准”。



标准按钮

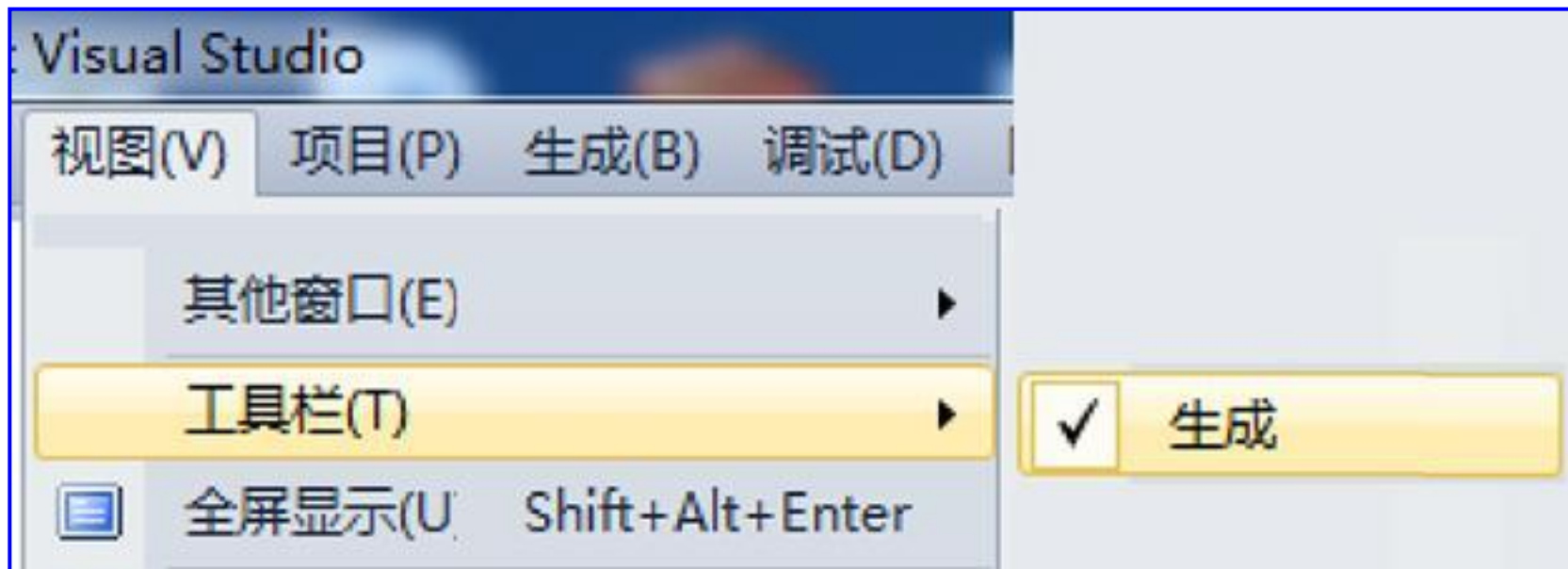


➤ 激活命令按钮的视图

菜单命令在VS中是固定的，标准按钮、编译按钮和调试按钮的视图可能在启动VS时没有出现，需要激活。

● 激活编译命令按钮的视图

用菜单命令选择“视图(V)”->“工具栏(T)”->“生成”。



编译按钮

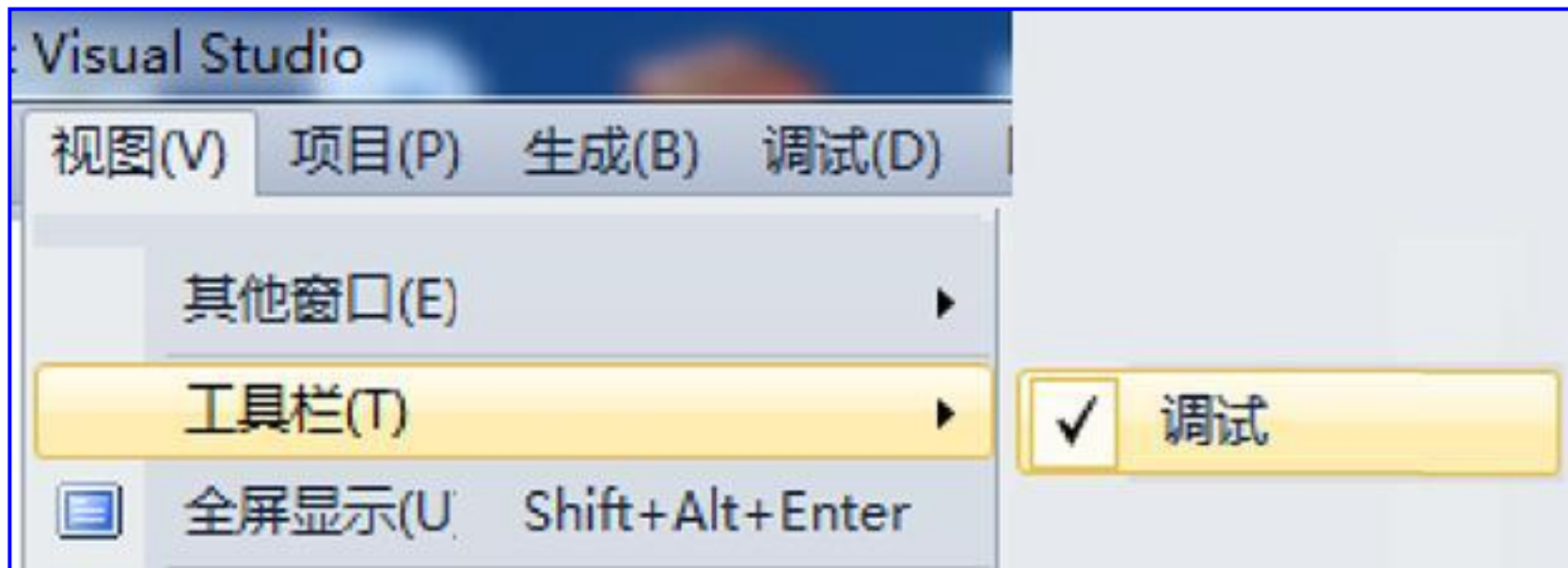


➤ 激活命令按钮的视图

菜单命令在VS中是固定的，标准按钮、编译按钮和调试按钮的视图可能在启动VS时没有出现，需要激活。

● 激活调试命令按钮的视图

用菜单命令选择“视图(V)”->“工具栏(T)”->“调试”。

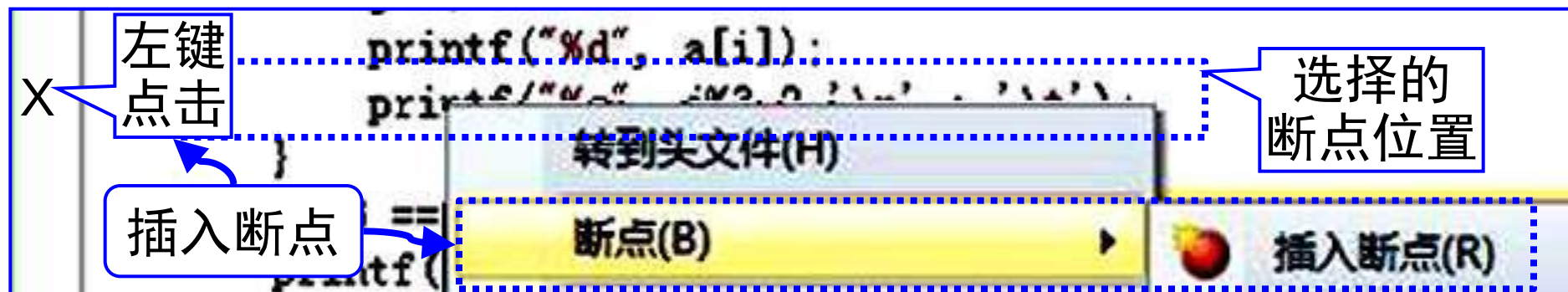


调试按钮

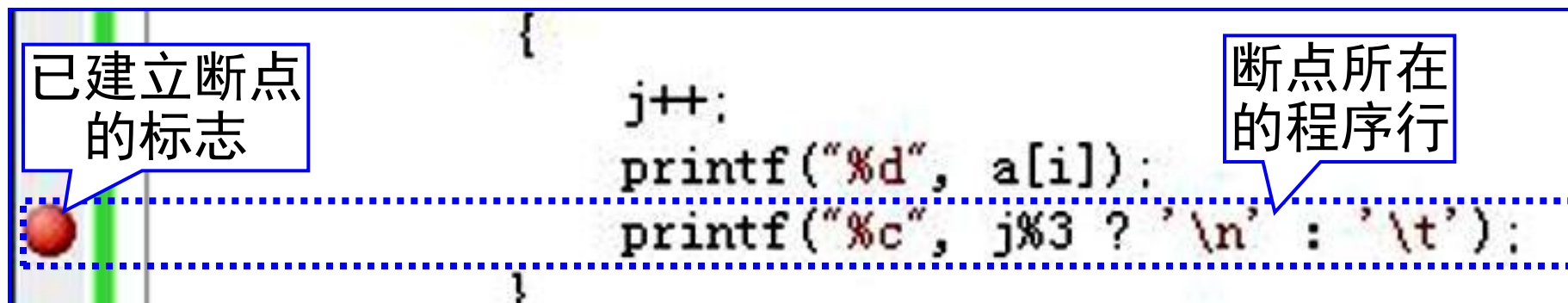


1) 建立(插入)断点

使光标位于将建立断点所在的程序行。可以选中该行中的任何字符，也可不选择任何字符。按右键选择“断点(B)”->“插入断点(R)”，或者左键单击断点行的行首。



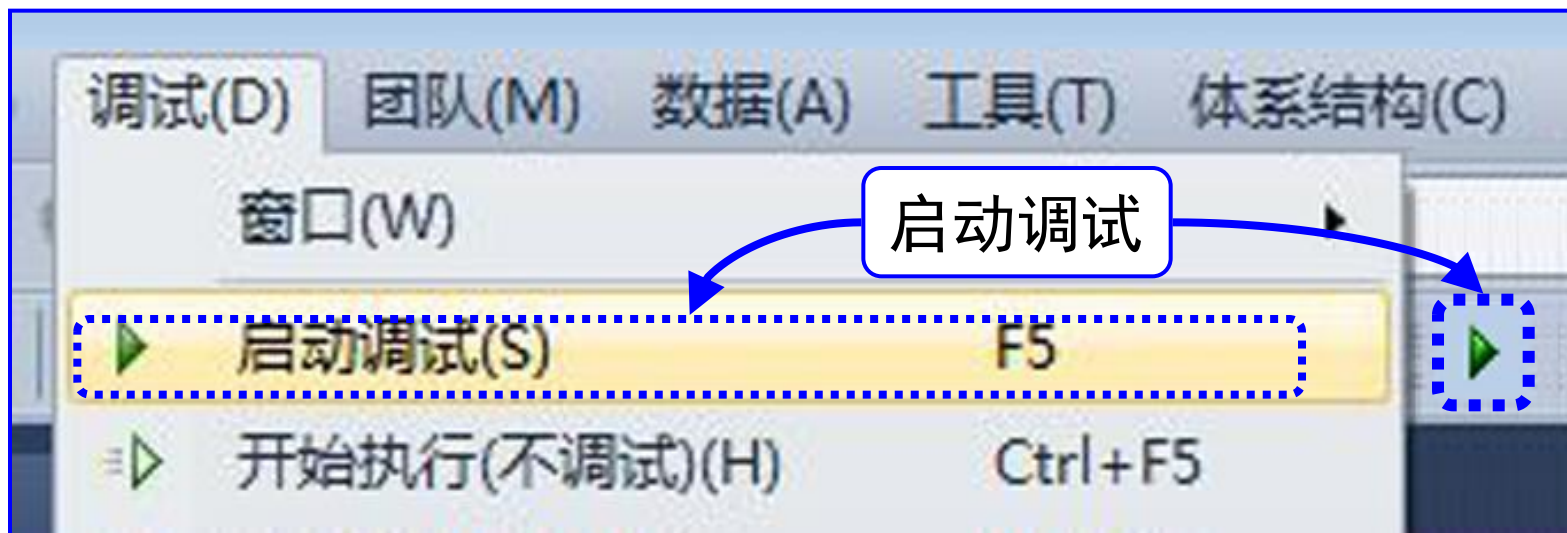
随后在断点所在的行首将出现一个圆圈，表示已建立了断点。



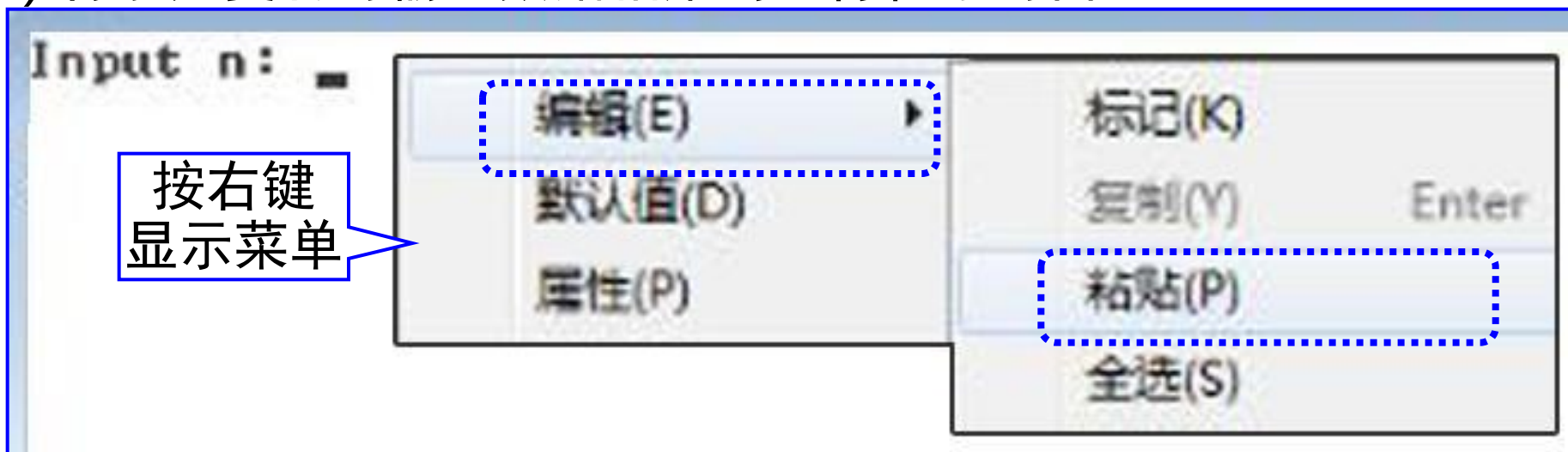
如果需要取消断点，重复操作一次。

2) 调试程序：运行

用菜单命令选择“调试(D)”->“启动调试(S)”，或用左键单击调试运行按钮”，或使用快捷键**F5**。

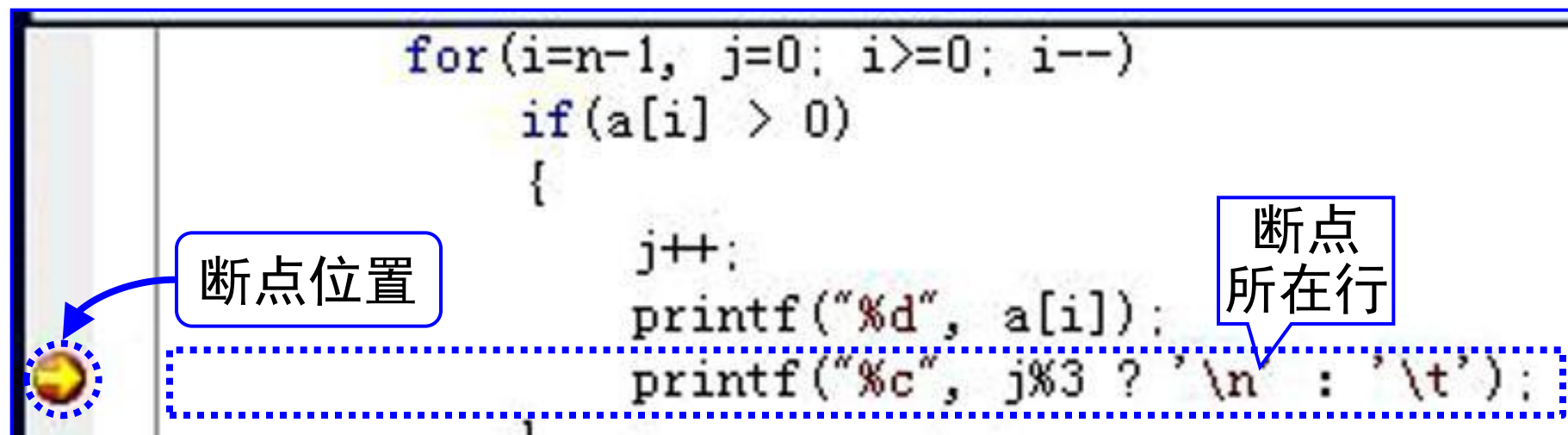


3) 将事先复制的输入数据粘贴到运行程序的窗口



4) 断点与程序运行的关系

程序获取数据后，将运行到断点位置停下，同时在表示断点位置的圆圈内显示黄色箭头(如果程序不是停在断点位置，则仅仅显示黄色的箭头)：



5) 观察变量的数值

在变量窗口、监视窗口以及程序运行的窗口观看变量的数值。

变量窗口显示当前涉及的变量及其数值。

通过拖动、键入等方式可以在监视窗口显示需要观看的变量或表达式及其数值。

观看变量窗口：

名称	值
 a[i]	2
 i	8
 j	1

自动窗口

监视1

观看监视窗口：

名称	值
 j % 3	1
 a[0]	1
 a[1]	0

键入变量或者表达式的位置

观看程序运行窗口：

Input n: 9
1 0 3 5 6 0 6 3 2
 ← 输出 2

断点位置



```
for(i=n-1, j=0; i >= 0; i--)  
{  
    if(a[i] > 0)  
    {  
        j++;  
        printf("%d", a[i]);  
        printf("%c", j%3 ? '\n' : '\t');  
    }  
}
```

断点所在行

6) 显示变量或表达式数值的方法

- 将光标放在选中的变量处(用鼠标刷一下)。
- 选中某个变量或表达式，将其拖到监视窗口。
- 在监视窗口键入变量或表达式。

```
for(i=n-1, j=0; i>=0; i--)  
    if(a[i] > 0)  
    {  
        a[i]=0  
        j++;  
        printf("%d", a[i]);  
        printf("%c", j%3 ? '\n' : '\t');
```

将光标放在
选中的变量处

a[i]=0

监视窗口：

名称	值
j % 3	1
a[0]	1
a[1]	0
a[1]+2	2

选中变量或表达式
将其拖到监视窗口

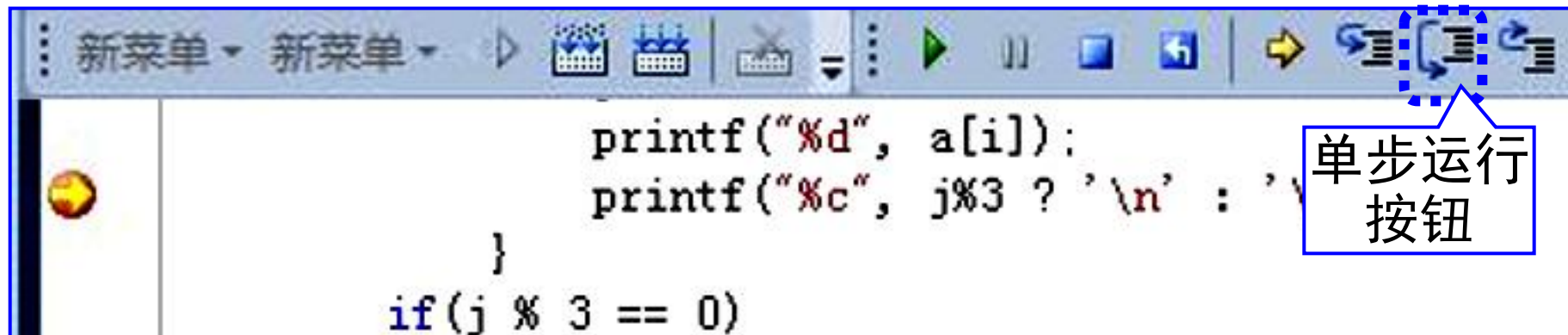
按右键，可以改为
十六进制显示

键入
表达式

7) 单步运行

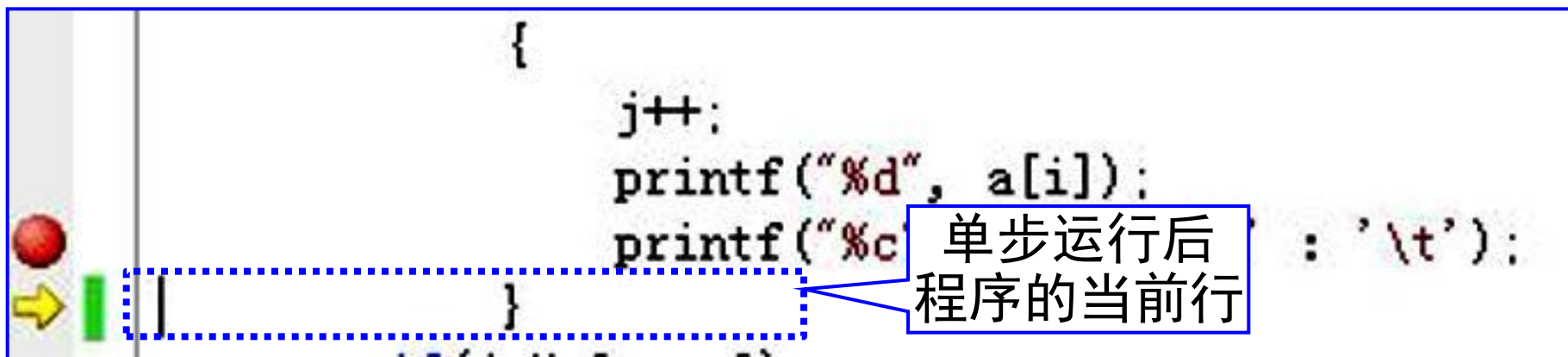
用左键单击单步运行按钮  ”，或使用快捷键**F10**。

不要使用左键单击单步运行按钮  ”，不要使用快捷键**F11**，以免进入printf的函数内部，无法调试。



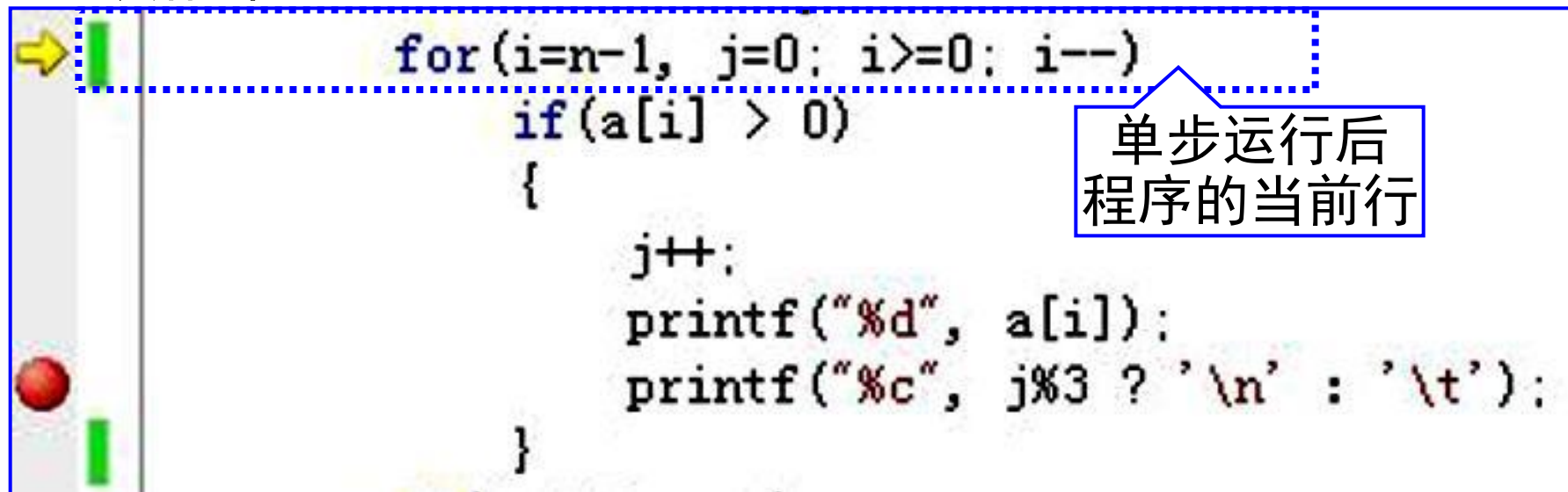
单步运行后，程序停在printf语句后if语句的“}”。

如果printf语句的后面还有其他语句，将停在下一语句所在的行。



7) 单步运行

再执行一次单步运行。程序停在for语句，表示将执行for语句的
下一次循环。

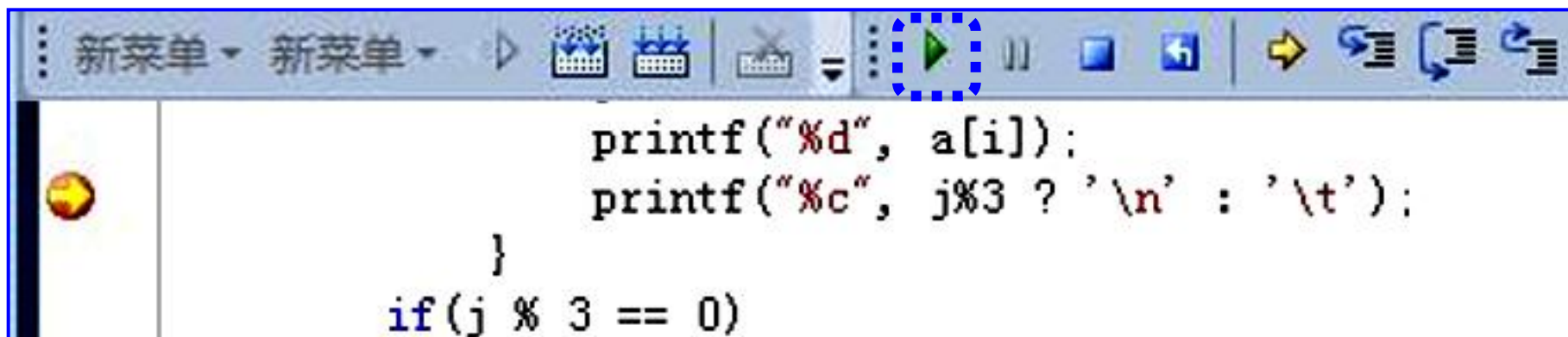


```
for(i=n-1, j=0; i>=0; i--)  
    if(a[i] > 0)  
    {  
        j++;  
        printf("%d", a[i]);  
        printf("%c", j%3 ? '\n' : '\t');  
    }
```

单步运行后
程序的当前行

8) 继续运行

再单击调试运行按钮  ”，程序又停在断点的位置。



```
printf("%d", a[i]);  
printf("%c", j%3 ? '\n' : '\t');  
}  
if(j % 3 == 0)
```

9) 再观察变量的数值

观看变量窗口：

名称	值
 a[i]	3
 i	7
 j	2

观看监视窗口：

名称	值
 j % 3	2
 a[0]	1
 a[1]	0

观看程序运行窗口：

Input n: 9
1 0 3 5 6 0 6 3 2
2
3

输出换行符

其中，在变量窗口和监视窗口中红色的数据表示经过最近的运行，这些数据刚刚发生了变化。

由于在监视窗口中发现当j=2时，j%3=2，此时应该输出空格而不是换行符，可以判断出是产生换行的语句发生了错误。

因此，应该将

```
printf("%c", j%3 ? '\n' : '\t');
```

改为

```
printf("%c", j%3 ? '\t' : '\n');
```

10) 修改程序，存盘，编译，然后重新调试运行。

11) 请同学们继续调试，改正输出数据的错误。

➤常用的调试命令

菜单命令	快捷键	按钮	命令功能
Executive	Ctrl+F5		运行程序(不调试)
Break Execution	Ctrl+Alt+Break		结束运行程序(不调试)
Go	F5		启动调试程序
Restart	Ctrl+Shift+F5		重新启动调试程序
Stop Debug	Shift+F5		结束调试程序
Step Into	F11		单步执行，或进入被调函数
Step Over	F10		单步执行(跨过被调函数)
Step Out	Shift+F11		跳出当前函数，返回主调函数

➤ 上机实习题

- 题源文件

debug01.rar

- 编程要求

程序功能：将文本文件中的各行按字典升序排序，逐行输出。

"lineSortKey.exe"运行示例1：

input file name: inFileA.txt

output file name: outFileA.txt

"lineSortKey.exe"运行示例2：

input file name: inFileB.txt

output file name: outFileB.txt

阅读"inFileA"和"outFileKeyA", "inFileB"和"outFileKeyB", 体会程序功能。

补齐"lineSort.c"中缺失的代码，提交该文件。

- 编程提示

请用strcmp函数比较各行文本的先后顺序。

➤ 上机实习题

● 程序调试要求

以下是学生提交的一些错误程序：

lineSort0A.c, lineSort1A.c, ..., lineSort9A.c

lineSort0B.c, lineSort1B.c, ..., lineSort9B.c

请你根据学号尾数，调试其中两个错误的C程序。

例如，学号尾数为0的同学，调试lineSort0A.c和lineSort0B.c。

提交word文件：debug01调试报告

报告内容：简要地写出调试两个程序发现的问题，以及调试程序给你的启发或者体会。